

컴퓨터 그래픽스 핵심 개념 정리 노트

1 장 10 장 강의 요약 + TA 실무 가이드

대상: 컴퓨터그래픽스 강의 TA 및 수강생

범위: 그래픽스 개관 - 기본요소 - 속성 - 기하변환 -
2D/3D 뷰잉 - 선과 면의 표현 - 가시면 결정 - 조명과 렌더링

본 노트는 강의 교재 PDF(1 장 10 장) 를 기반으로 개념을 재구성하고,
TA 로서 실습 지도·과제 채점·시험 출제 시 참고할 실무 포인트를 추가하였습니다.

Contents

1	그래픽스 개관과 소프트웨어 (1 2 장)	3
1.1	컴퓨터 그래픽스의 기본 개념	3
1.2	래스터 그래픽스 vs 벡터 그래픽스 영상	3
1.3	그래픽스 소프트웨어의 두 층위	3
1.4	OpenGL 과 그래픽스 파이프라인	3
2	컴퓨터 그래픽스의 기본 요소 (3 장)	4
2.1	동차좌표 예고	4
2.2	선분 래스터화 알고리즘	4
2.3	원과 다른 곡선	5
2.4	다각형	5
3	그래픽스 기본 요소의 속성 (4 장)	6
3.1	색 모델	6
3.2	OpenGL 색 함수	7
3.3	점·선 속성	7
3.4	영역 채우기	7
3.5	안티에일리어싱	8
4	기하변환 (5 장)	8
4.1	2 차원 기본 변환	8
4.2	동차좌표와 행렬표현	8
4.3	3 차원 기본 변환	9
4.4	복합변환	9
4.5	기타 변환 및 아핀 변환	9
4.6	좌표계 간 변환	9
4.7	OpenGL 에서의 기하변환	9
5	2 차원 뷰잉 (6 장)	10
5.1	뷰잉 파이프라인	10
5.2	클리핑 원도 → 뷰포트 변환	10
5.3	점 클리핑 조건	10
5.4	선분 클리핑	10
5.5	다각형 클리핑	11
6	3 차원 뷰잉 (7 장)	11
6.1	뷰잉 파이프라인과 카메라 비유	12
6.2	뷰잉 좌표 파라미터	12
6.3	평행투영	12
6.4	원근투영	12
6.5	정규화와 뷰 체적	13
7	선과 면의 표현 (8 장)	13
7.1	3 차원 객체 모델링 방식	13
7.2	2 차 곡면	13
7.3	스플라인 곡선 개요	13
7.4	에르미트 (Hermite) 스플라인	14
7.5	베지어 (Bézier) 스플라인	14
7.6	B-스플라인	14
7.7	NURBS (비균일 유리 B-스플라인)	15
7.8	스플라인 곡면과 테셀레이션 셰이더	15

8	가시면 결정 (9 장)	15
8.1	개념	15
8.2	후면 제거 (back-face culling)	16
8.3	깊이 버퍼 (Z-버퍼) 알고리즘	16
8.4	주사선 (scan-line) 알고리즘	16
8.5	깊이정렬 (화가의 알고리즘)	16
8.6	OpenGL 관련 함수	17
9	조명 및 표면 렌더링 (10 장)	17
9.1	표면 렌더링 개요	17
9.2	조명 모델	17
9.3	광원의 종류	17
9.4	표면 반사	18
9.5	퐁 (Phong) 반사 모델 종합식	18
9.6	다각형 음영법	18
10	TA 실무 가이드	20
10.1	실습 (OpenGL) 지도 시 체크리스트	20
10.2	과제·시험 채점 시 챕터별 확인 포인트	20
10.3	학생들이 가장 자주 헷갈리는 개념 Top 10	21
10.4	현업·최신 그래픽스와의 연결 고리	21
10.5	시험·퀴즈 출제 시 유의점	21
10.6	핵심 알고리즘 총정리 카드	22

1 그래픽스 개관과 소프트웨어 (1 2 장)

1.1 컴퓨터 그래픽스의 기본 개념

컴퓨터 그래픽스란 컴퓨터에 의한 그림 자료의 표현과 조작을 다루는 컴퓨터과학의 한 분야이다. 컴퓨터 영상을 만드는 방법은 크게 (1) 태블릿·펜을 이용한 2D 래스터 그래픽, (2) 가상환경을 이용한 3D 벡터 그래픽, (3) 프로그램을 이용한 생성으로 나뉜다. 그래픽스 시스템은 **그래픽스 워크스테이션** (CPU + 메모리 + I/O 인터페이스 + GPU) 과 입출력 장치로 구성되며, GPU 는 그래픽스 렌더링에 필요한 연산을 가속하기 위해 특화된 프로세서이다.

디스플레이 방식은 크게 두 가지이다.

- **벡터 디스플레이**: 그림을 그래픽스 명령 (디스플레이 리스트) 형태로 저장, 재생 버퍼의 내용을 프로세서가 초당 30 회 이상 반복 출력. 리스트가 커지면 깜빡임 발생.
- **래스터 디스플레이**: 전자빔이 좌 → 우, 위 → 아래로 주사하며 화면을 픽셀 단위로 채움. 반복 빈도가 **주사율**, 주사선당 픽셀 수 × 주사선 수가 **해상도**.

해상도가 낮으면 직선이 계단 모양으로 보이는 **에일리어싱 (aliasing)** 현상이 발생하며, 이를 완화하는 기법이 **안티에일리어싱**이다 (4 장에서 상세). 기하 도형을 이산적인 픽셀 집합으로 바꾸는 과정을 **주사변환 (스캔 컨버전, 래스터화)** 이라 한다.

디스플레이 품질 지표 해상도 (분해능) / 도트 피치 (작을수록 선명) / 반응시간 (LCD, ms 단위) / 재생률 / 지원 색상 수 / 시야각 / 명암비 / 종횡비

1.2 래스터 그래픽스 vs 벡터 그래픽스 영상

- **래스터 영상**: 사각 격자의 픽셀 배열로 표현, 확대 시 화질 저하 (BMP, PNG, JPEG 등). Photoshop 계열.
- **벡터 영상**: 점·선·곡선·다각형 등 기하 정보만 저장하여 용량이 작고 확대해도 화질 저하 없음 (SVG, DXF, STL 등). Illustrator 계열이며 최종 출력 시 래스터화 과정을 거침.

1.3 그래픽스 소프트웨어의 두 층위

- **특수목적 그래픽스 패키지**: CAD, Photoshop, Maya 등—비전문가도 사용 가능한 완성된 응용.
- **범용 그래픽스 API**: OpenGL, DirectX 등—프로그래머가 그래픽스 함수를 호출하여 응용을 개발.
 - **저수준 API**: 점·선·다각형 등 기본 요소와 색, 기하변환, 뷰잉 등을 일일이 지정 (OpenGL, DirectX).
 - **고수준 API**: 장면 객체를 계층적 모형으로 다루어 세부 처리를 감춤 (Open Inventor, VRML, Java 3D).

1.4 OpenGL 과 그래픽스 파이프라인

OpenGL 은 플랫폼 독립적인 저수준 절차적 API 로, 명령 이름은 gl 접두어 + 낙타 표기 (예: glClearColor), 상수는 GL_ 접두어의 대문자 (예: GL_COLOR_BUFFER_BIT) 를 사용한다. OpenGL 2.0 부터 고정기능 파이프라인 대신 **셰이더**가 동작하는 **프로그램 가능 파이프라인**이 도입되었고, 셰이더는 GLSL 로 작성한다.

OpenGL 파이프라인 단계 (필수 단계는 굵게 표시):

1. Vertex Puller (꼭짓점 데이터 수집)
2. **Vertex Shader** —정점 단위 처리 (필수)
3. Tessellation Control/Evaluation Shader —기하구조 세분화 (선택, OpenGL 4.0+)

4. Geometry Shader —기본 요소 단위 수정 (선택)
5. Rasterization —래스터화 (고정기능)
6. **Fragment Shader** —픽셀 (프래그먼트) 단위 색상·깊이 계산 (필수)
7. 프레임 버퍼 동작

셰이더 프로그램 준비 절차 (실습에서 매번 반복되는 패턴):

1. `glCreateShader` → `glShaderSource` → `glCompileShader` → `glGetShaderiv(GL_COMPILE_STATUS)` 로 컴파일 확인
2. `glCreateProgram` → `glAttachShader` (정점·프래그먼트 각각)
3. `glLinkProgram` → `glGetProgramiv(GL_LINK_STATUS)` 로 링크 확인
4. `glValidateProgram` → `glUseProgram`

그리기 준비: `glGenBuffers` 로 VBO 핸들 생성 → `glBindBuffer(GL_ARRAY_BUFFER, ...)` → `glBufferData` 로 정점 데이터 전달 → 렌더링 시 `glEnableVertexAttribArray + glVertexAttribPointer` 로 속성 위치 지정 → `glDrawArrays(mode, first, count)` 호출.

TA 노트 1-2 장은 시험보다는 실습 환경 세팅과 직결됩니다. 학생들이 처음 겪는 오류 대부분은 (1) GLEW/freeGLUT 초기화 순서 (반드시 `glutCreateWindow` 이후에 `glewInit`), (2) 셰이더 컴파일 오류 로그를 확인하지 않고 넘어가는 것, (3) VBO 를 만들고도 `glVertexAttribPointer` 의 stride/offset 을 잘못 계산하는 것입니다. 실습 첫 시간에는 셰이더 컴파일·링크 실패 시 반드시 `glGetShaderInfoLog`/`glGetProgramInfoLog` 를 출력하도록 습관화시키는 것이 이후 모든 실습의 디버깅 속도를 좌우합니다.

자주 하는 실수 `glVertexAttribPointer` 의 stride 에 구조체 전체 크기 (`sizeof(Vertex)`) 를 넣지 않고 개별 필드 크기만 넣어 정점마다 색이 밀려 보이는 오류, 그리고 색 속성의 offset 에 (`const GLvoid*`) 형변환을 빠뜨려 컴파일 경고를 무시하고 넘어가는 경우가 매우 흔합니다.

2 컴퓨터 그래픽스의 기본 요소 (3 장)

2.1 동차좌표 예고

3 장부터 셰이더 코드에서 `vec4(Position, 1.0)` 처럼 좌표에 네 번째 성분 h 를 붙이는 것을 볼 수 있는데, 이는 5 장에서 다루는 **동차좌표**의 사용 예이다. 데카르트 좌표 (x, y, z) 는 동차좌표 (hx, hy, hz, h) 로 표현되며 $h = 1$ 이 가장 단순한 선택이다.

2.2 선분 래스터화 알고리즘

래스터 화면은 정수 좌표만 존재하므로 실수 좌표는 가장 가까운 정수 좌표로 반올림된다 (선분의 래스터화). 직선의 기울기-절편 방정식 $y = mx + b$, $m = \Delta y / \Delta x$ 에서 출발한다.

(1) DDA(Digital Differential Analyzer) 알고리즘

차이가 더 큰 축을 1 씩 증가시키며 다른 축 값을 증분으로 계산한다.

$$x_{k+1} = x_k + x_{Increment}, \quad y_{k+1} = y_k + y_{Increment}$$

장점: 곱셈이 없어 직접 계산보다 빠름. 단점: 실수 연산·반올림이 필요하여 긴 선분에서 오차가 누적됨.

(2) 브레젠햄 (Bresenham) 알고리즘—중간점 알고리즘

실수 연산을 완전히 제거하고 **정수 덧셈만으로** 다음 픽셀을 결정한다. 기울기 $0 < m < 1$ 구간에서 두 후보 픽셀의 중간점 $(x_k + 1, y_k + 0.5)$ 에서 판별함수

$$F(x, y) = 2H(x - x_1) - 2W(y - y_1), \quad H = y_2 - y_1, \quad W = x_2 - x_1$$

값의 부호로 다음 픽셀을 결정한다. 판별식은 상수 $2H$ 또는 $2(H - W)$ 만 누적해서 더하면 되므로 매우 빠르다.

- $F(M_{k+1}) < 0$: y 그대로, $F += 2H$
- $F(M_{k+1}) \geq 0$: y 를 1 증가, $F += 2(H - W)$

8 분면·4 분면 대칭성을 이용해 임의의 기울기로 일반화한다. 원, 곡선 알고리즘의 원형이 되는 중요한 아이디어이다.

TA 노트 Bresenham/중간점 알고리즘은 시험·과제에서 가장 자주 나오는 계산 문제입니다. 채점 시 다음을 확인하세요: (1) 기울기 부호별 8 가지 케이스 (수평선·수직선·45° 포함) 처리 여부, (2) 판별식 초기 값 계산이 정확한지, (3) 정수 연산만 사용했는지 (실수 연산이 섞이면 감점 포인트). 학생들은 $|m| > 1$ 인 경우 x, y 역할을 바꿔야 한다는 점을 자주 놓칩니다.

2.3 원과 다른 곡선

원의 방정식 $(x - x_c)^2 + (y - y_c)^2 = r^2$ 을 직접 풀면 제곱근 연산이 필요하고 y 간격이 불균일해 비효율적이다. **중간점 원 알고리즘**은 원 함수

$$F(x, y) = x^2 + y^2 - r^2$$

을 이용, 원 내부는 음수·외부는 양수임을 이용해 8 분면 대칭으로 계산량을 1/8 로 줄인다. 판별식 초기값은 정수인 경우 $1 - r$ 을 사용한다 (부호만 중요하므로 $1.25 - r$ 의 0.25 상수항 생략).

타원 $\left(\frac{x - x_c}{r_x}\right)^2 + \left(\frac{y - y_c}{r_y}\right)^2 = 1$ 은 4 분면 대칭만 가능하다. 포물선 $y = a(x - m)^2 + n$, 쌍곡선은 타원 방정식과 부호만 다르다. 일반 원뿔곡선 $Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$ 은 판별식 $B^2 - 4AC$ 의 부호로 종류가 결정된다 (< 0 : 타원, $= 0$: 포물선, > 0 : 쌍곡선).

2.4 다각형

와인딩 순서와 앞면/뒷면

꼭짓점을 나열하는 순서를 **와인딩 순서**라 하며, 앞면을 바라볼 때 반시계 방향 (CCW) 으로 나열하면 CCW 와인딩이다. OpenGL 은 기본적으로 CCW 를 **앞면으로 간주**한다 (9 장 후면 제거와 직결).

신발끈 공식 (Shoelace formula) 으로 CCW 다각형의 면적을 구할 수 있다.

$$\text{area} = \frac{1}{2} \sum_{k=0}^{n-1} x_k (y_{(k+1) \bmod n} - y_{(k-1) \bmod n})$$

CW 로 나열되면 면적이 음수로 계산되어, 이 부호로 앞면/뒷면을 판별할 수 있다.

블록·오목 판별

인접 꼭짓점을 잇는 벡터 쌍의 외적을 모두 계산했을 때 방향이 전부 같으면 블록다각형, 그렇지 않으면 오목다각형이다.

내부·외부 판별: 홀·짝 규칙 (odd-even rule)

임의의 점에서 외부 기준점까지 판별선을 긋고 다각형 변과의 교차 횟수를 센다. **교차 횟수가 홀수면 내부, 짝수면 외부**. 판별선이 **꼭짓점을 지나는 경우가 함정이다**—꼭짓점을 공유하는 두 변이 판별선 양쪽에 나뉘어 있으면 1 회 교차로, 한쪽에 모여 있으면 0 회 또는 2 회 교차로 처리해야 정확하다.

자주 하는 실수 점-다각형 내부 판별 알고리즘을 구현할 때 꼭짓점을 지나는 경우와 수평 변을 특별 처리하지 않으면 경계 근처에서 오류가 발생합니다. 이는 4 장의 주사선 다각형 채우기와 정확히 동일한 문제이므로, 두 알고리즘을 연결지어 가르치면 학생들의 이해도가 높아집니다.

앞면 판별 (3 차원)

평면의 법선 벡터 $N = E_1 \times E_2$ (와인딩 순서에 따라 연결한 두 변 벡터의 외적) 로 계산한다. 점 (x, y, z) 가 평면 $Ax + By + Cz + D = 0$ 의 앞/뒤에 있는지는 $Ax + By + Cz + D$ 의 부호로 판별한다 (9 장 후면 제거의 기초).

다각형 표 (Polygon Table)

장면의 기하구조는 **꼭짓점 표 / 변 표 / 면 표**의 3 개 리스트로 관리한다. 변 표는 꼭짓점 표를 참조하고, 면 표는 변 표를 참조하는 계층 구조로, 인접 면을 공유하는 변을 효율적으로 식별할 수 있다.

OpenGL 삼각형 그리기

OpenGL 코어 프로파일은 임의의 다각형을 지원하지 않고 **삼각형만** 지원한다 (항상 평면·볼록 보장). GL_TRIANGLES(3 개씩 독립), GL_TRIANGLE_STRIP(연속 공유), GL_TRIANGLE_FAN(첫 꼭짓점 중심 부채꼴) 의 세 방식이 있다.

TA 노트 "OpenGL 은 왜 삼각형만 그리나요?" 는 학생들이 자주 하는 질문입니다. 답은 (1) 3 점은 항상 한 평면 위에 있고, (2) 항상 볼록하여 래스터화 알고리즘이 단순·고속화되기 때문입니다. 오목다각형이나 임의의 다각형은 반드시 삼각형화 (triangulation) 해서 넘겨야 한다는 점을 강조하면 좋습니다—이는 실무의 메시 처리 (예: Unity 의 MeshFilter, glTF 포맷) 에서도 동일하게 요구되는 전처리 과정입니다.

3 그래픽스 기본 요소의 속성 (4 장)

3.1 색 모델

색의 3 요소: **색조 (hue)**(지배 주파수), **채도 (saturation)**(스펙트럼색에 가까운 정도), **명도 (brightness)**(에너지 총량). **삼원자극이론**: 인간 망막의 세 원추세포 (약 440/545/580nm 에서 최대 흡수) 가 감지하는 빛의 양이 같으면 스펙트럼 구성이 달라도 같은 색으로 인지된다. RGB 삼원색으로 표현 가능한 색의 집합을 **색역 (gamut)** 이라 하며, 이는 사람이 볼 수 있는 가시광선 색의 일부만 포함한다. CIE 는 실존하지 않는 가상의 표준원색 X, Y, Z 를 정의하여 (1931) 평균적 인간이 보는 모든 색을 수학적으로 표현할 수 있게 하였다 (Y 가 밝기).

모델	원색	유형	용도
RGB	빨강·초록·파랑	가산색 (빛을 더함)	모니터, LED, 발광 디스플레이
CMY(K)	청록·자홍·노랑 (+ 검정)	감산색 (빛을 흡수)	프린터 등 반사형 출력
HSV	색조·채도·명도	직관적 모델 (육각뿔)	UI 색상 선택기

RGB↔CMY 변환: $C = 1 - R$, $M = 1 - G$, $Y = 1 - B$ (역도 동일).

프레임 버퍼 색 저장 방식 두 가지: (1) RGB 부호를 프레임 버퍼에 **직접 저장** (참색/풀컬러는 채널당 8 비트, 총 24 비트 + 알파 8 비트 = 32 비트), (2) **컬러 룩업 테이블 (색 인덱스) 방식**—256 색 팔레트 시절 널리 쓰였으나 현재는 대부분 직접 저장 방식을 사용.

3.2 OpenGL 색 함수

정점 색은 $[0,1]$ 범위 GLfloat 또는 정수형 (정규화되어 셰이더로 전달) 으로 지정한다. **색 혼합 (블렌딩)** 은

$$(R'_d, G'_d, B'_d, A'_d) = (S_r R_s + D_r R_d, S_g G_s + D_g G_d, S_b B_s + D_b B_d, S_a A_s + D_a A_d)$$

로 계산되며, glBlendFunc(sFactor, dFactor) 로 혼합 인수를 지정한다 (GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA 조합이 표준 알파 블렌딩).

3.3 점·선 속성

점은 glPointSize, 선은 glLineWidth 로 크기를 지정한다. 표준 선 폭 구현은 기울기에 따라 상·하 또는 좌·우로 픽셀을 추가하는 방식이라 **기울기에 따라 선이 가늘어 보이는 문제와 끝이 사선으로 잘리는 문제**가 있다. GL_LINE_SMOOTH 를 활성화하면 선분 폭 만큼의 직사각형 영역을 채워 기울기 무관하게 균일한 폭을 얻는다. 선 스타일 (실선/점선/파선) 은 래스터화 시 일정 구간 픽셀을 건너뛰어 구현한다.

3.4 영역 채우기

비트맵 패턴 타일 붙이기

직사각형 마스크를 반복 복제 (타일링) 하여 채운다. 기준점을 **원도 원점**으로 하면 인접 다각형의 패턴이 이어져 보이나 (애니메이션 시 패턴이 밀리는 단점), **도형의 경계상자 모서리**를 기준으로 하면 다각형마다 독립적으로 채워진다.

주사선 다각형 채우기 알고리즘

각 주사선에서 다각형 변과의 교차점을 구해 **홀-짝 규칙**으로 내부 구간을 채운다. 정확한 처리 규칙:

1. 변의 두 끝점 중 y 값이 큰 꼭짓점은 그 변에 포함시키지 않는다 (경계 중복 방지).
2. x 축과 평행한 변은 계산에서 제외한다.
3. 교차 구간 $[x_1, x_2]$ 에서 $\lceil x_1 \rceil$ 부터 $\lfloor x_2 \rfloor - 1$ 까지를 내부 픽셀로 한다.

효율적 구현을 위해 **선분 테이블 (edge table)**(주사선별 버킷에 변 등록: $y_{\min}, y_{\max}, 1/m$, 포인터) 과 **활동 선분 목록 (active edge table)**(현재 주사선과 교차 중인 변만 유지, x 증분 $1/m$ 을 매 주사선마다 누적) 을 사용한다.

불규칙 경계 채우기

- **경계선 채우기 (boundary fill)**: 지정된 경계색을 만날 때까지 재귀적으로 채움.
- **범람 채우기 (flood fill)**: 지정된 내부색과 같은 색인 동안 재귀적으로 채움 (페인트 통 도구의 원리).

4 방향/8 방향 연결의 선택은 경계선 정의 방식과 **반대로** 선택해야 한다 (4 방향 경계 → 8 방향 채우기, 8 방향 경계 → 4 방향 채우기). 그렇지 않으면 경계 누출 또는 미채움 문제가 발생한다.

자주 하는 실수 재귀적 경계선/범람 채우기는 큰 다각형에서 **스택 오버플로**가 발생하기 쉽습니다. 과제에서 학생들이 재귀 깊이 제한에 걸리는 경우가 많으므로, 스캔라인 기반 반복적 구현 (span-based flood fill) 을 소개하거나 스택 크기를 늘리는 방법을 안내하면 좋습니다.

3.5 안티에일리어싱

낮은 표본화 (저주파 표본화) 로 인한 정보 왜곡이 **에일리어싱**이다. 나이키스트 표본화 주파수 $f_s = 2f_{max}$ 이상으로 표본화해야 왜곡을 피할 수 있다.

기법	원리
영역 샘플링 (전 필터링)	물체가 차지하는 픽셀 면적 비율로 음영 조절
슈퍼 샘플링 (SSAA)	더 높은 해상도로 렌더링 후 평균 (정확하지만 매우 비쌈)
MSAA	다각형 경계 서브샘플만 판정, 픽셀당 음영 계산은 1 회 (SSAA 보다 저렴)
후 필터링 FXAA/MLAA	완성된 영상에 가중 평균 필터 적용 (간단하지만 번짐 발생) 에지 방향을 검출해 후처리로 필터링 (번짐 최소화, 실시간 게임에서 표준)

OpenGL에서는 `glEnable(GL_BLEND) + glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)` 후 `glEnable(GL_POINT_SMOOTH/LINE_SMOOTH/POLYGON_SMOOTH)` 로 안티에일리어싱을 적용한다.

TA 노트 안티에일리어싱은 실무 그래픽스 옵션 (게임 설정의 "MSAA 4x", "TAA", "FXAA") 과 바로 연결되는 대목입니다. 학생들에게 "왜 MSAA 가 SSAA 보다 빠른가" 를 물어보면 셰이딩 계산과 커버리지 (coverage) 샘플링을 분리해서 이해하는지 확인할 수 있는 좋은 질문이 됩니다. 최신 엔진 (Unreal, Unity URP/HDRP) 은 TAA(Temporal AA) 를 기본값으로 쓰는데, 이는 후 필터링 개념을 시간축으로 확장한 것이라는 점을 언급하면 이 장의 개념이 최신 기술과 어떻게 이어지는지 보여줄 수 있습니다.

4 기하변환 (5 장)

4.1 2 차원 기본 변환

$$\text{이동: } x' = x + t_x, y' = y + t_y$$

$$\text{크기: } x' = s_x x, y' = s_y y$$

$$\text{회전 (원점 기준, 반시계): } x' = x \cos \phi - y \sin \phi, y' = x \sin \phi + y \cos \phi$$

크기변환 요소가 1 보다 작으면 축소, 크면 확대, $s_x = s_y$ 이면 비율 유지. 임의 고정점 (x_f, y_f) 기준 크기변환은 $x' = s_x x + x_f(1 - s_x)$ (마찬가지로 y).

4.2 동차좌표와 행렬표현

기하변환의 연속 적용을 하나의 행렬 곱으로 통합하기 위해 **동차좌표** $(x, y, h) \rightarrow (x, y, 1)(2D)$, $(x, y, z, 1)(3D)$ 를 사용한다. 모든 변환을 $3 \times 3(2D)/4 \times 4(3D)$ 행렬로 표현할 수 있다.

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \quad S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

핵심 요약 그래픽스 라이브러리는 좌표를 **열벡터**로 표현하는 것이 표준이며 (OpenGL, Java 3D, PHIGS, GKS), 변환행렬은 열벡터 **왼쪽**에 곱한다: $P' = MP$. 여러 변환을 연속 적용하면 $P' = M_n \cdots M_2 M_1 P$ 이며, **가장 오른쪽 행렬이 가장 먼저 적용된다**.

4.3 3 차원 기본 변환

3 차원은 4×4 동차좌표 행렬로 확장된다. 축 기준 회전:

$$R_z(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi & 0 & 0 \\ \sin \phi & \cos \phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi & 0 \\ 0 & \sin \phi & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_y(\phi) = \begin{bmatrix} \cos \phi & 0 & \sin \phi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \phi & 0 & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

회전행렬의 역행렬은 전치행렬과 같다 ($R^{-1} = R^T$, $\phi \rightarrow -\phi$ 와 동일).

4.4 복합변환

행렬 곱셈은 **결합법칙**은 성립하지만 **교환법칙**은 성립하지 않으므로 순서를 지켜야 한다. 대표 패턴:

- 고정점 기준 크기변환 = 이동 ($-x_f, -y_f$) $\rightarrow$ 크기변환 $\rightarrow$ 이동 (x_f, y_f)
- 임의 중심점 기준 회전 = 이동 ($-x_r, -y_r$) $\rightarrow$ 회전 $\rightarrow$ 이동 (x_r, y_r)
- 임의 회전축 기준 3D 회전 (5 단계): 이동 (축을 원점 통과) $\rightarrow x$ 축 정렬 회전 $\rightarrow y$ 축 정렬 회전 (z 축과 일치) $\rightarrow z$ 축 중심 실제 회전 $\rightarrow$ 앞의 회전들을 역순 역변환으로 복원

4.5 기타 변환 및 아핀 변환

대칭변환 (거울 반사), **밀림 (shear) 변환** (한 축이 다른 축 값에 비례해 선형 변화) 도 3×3 행렬로 표현된다. 이동·회전·크기·대칭·밀림 변환은 모두 **아핀 변환**이며, 아핀 변환은

- 동일 직선 위 세 점은 변환 후에도 동일 직선 위에 있다.
- 한 직선 위 점들 사이 거리의 비가 보존된다.
- 평행선은 변환 후에도 평행하다.

이동·회전·대칭만으로 구성된 아핀 변환은 물체 내 모든 점 쌍의 거리를 보존하는데 이를 **강체변환 (rigid-body transformation)** 이라 한다.

자주 하는 실수 원근투영은 아핀 변환이 아니다. 이 사실은 7 장 (뷰잉) 과 8 장 (NURBS) 두 곳에서 다시 등장하는 핵심 포인트로, "왜 베지어/비유리 B-스플라인은 원근투영 후 곡선이 어긋날 수 있는가" 라는 질문의 답이 됩니다. 5 장을 가르칠 때 이 사실을 미리 언급해 두면 8 장에서 NURBS의 필요성을 설명하기 훨씬 쉬워집니다.

4.6 좌표계 간 변환

한 좌표계 ($x-y-z$) 에서 정의된 물체를 새로운 좌표계 ($x'-y'-z'$, 단위좌표벡터 u_x, u_y, u_z) 로 옮기는 변환은 (1) 새 좌표계 원점을 기존 원점으로 이동, (2) 두 좌표축을 일치시키는 복합 회전으로 구성된다. 이는 7 장 뷰잉 변환의 직접적인 기초가 된다.

4.7 OpenGL 에서의 기하변환

변환행렬은 셰이더의 uniform mat4 변수로 전달한다: glGetUniformLocation 으로 위치를 얻고 glUniformMatrix4fv(location, count, transpose, value) 로 전달한다. transpose 인자는 행렬이 **열 우선 (column-major)** 으로 저장되어 있으면 GL_FALSE, **행 우선 (row-major)** 이면 GL_TRUE 를 지정해야 한다.

TA 노트 실습에서 가장 많이 나오는 버그가 바로 `glUniformMatrix4fv` 의 `transpose` 인자입니다. C 배열로 행렬을 `mat[row][col]` 형태 (행 우선) 로 만든 경우 `GL_TRUE` 를 넘겨야 하는데, OpenGL 내부 관례 (열 우선) 에 익숙한 학생이 무심코 `GL_FALSE` 를 넣어 변환 결과가 전치되어 물체가 이상하게 찌그러지는 문제가 자주 발생합니다. 코드를 리뷰할 때는 행렬을 구성하는 방식과 `transpose` 인자가 일관되는지 항상 함께 확인하세요. 또한 복합변환 순서 (예: $S \rightarrow R \rightarrow T$ vs $T \rightarrow R \rightarrow S$) 를 반대로 곱해 결과가 달라지는 실수도 매우 흔하므로, "가장 나중에 적용하고 싶은 변환을 가장 왼쪽에 곱한다" 는 규칙을 반복해서 상기시켜 주는 것이 좋습니다.

5 2차원 뷰잉 (6 장)

5.1 뷰잉 파이프라인

장면 중 화면에 표시할 영역을 **클리핑 윈도**, 그 내용이 표시될 출력 영역을 **뷰포트**라 한다. 클리핑 윈도 밖은 잘려나간다 (클리핑). 파이프라인:

세계좌표 장면 구성 → 뷰잉 좌표 변환 → 정규화 좌표 변환 (+ 클리핑) → 장치좌표 (뷰포트) 변환

클리핑 윈도와 뷰포트의 상대적 크기/위치를 조정하면 확대 (zoom, 뷰포트 고정 + 윈도 축소), 이동 (pan, 윈도 크기 고정 + 위치 이동) 등의 효과를 낼 수 있다.

5.2 클리핑 윈도 → 뷰포트 변환

클리핑 윈도를 $[-1, 1]^2$ 정규화 좌표로 보내는 변환은 (윈도 최소점 기준 크기변환) → (이동) 의 복합이며, 정규화 좌표를 다시 뷰포트로 보내는 변환도 동일한 패턴이다. 최종 변환행렬은

$$M_{viewport \leftarrow window} = T(vx_{min}, vy_{min}) \cdot S\left(\frac{vx_{max} - vx_{min}}{wx_{max} - wx_{min}}, \frac{vy_{max} - vy_{min}}{wy_{max} - wy_{min}}\right) \cdot T(-wx_{min}, -wy_{min})$$

과 같이 이동 → 크기 → 이동 세 단계의 합성으로 유도할 수 있다 (5 장 "고정점 기준 변환" 패턴의 응용).

5.3 점 클리핑 조건

$$wx_{min} \leq x \leq wx_{max}, \quad wy_{min} \leq y \leq wy_{max}$$

넷 중 하나라도 불만족하면 클리핑 윈도 외부.

5.4 선분 클리핑

(1) 코헨-서덜랜드 (Cohen-Sutherland)

윈도 주변을 9 개 영역으로 나누고 각 끝점에 4 비트 **영역 코드** (상/하/우/좌 위반 여부) 를 부여한다.

- $RC_1 \text{ OR } RC_2 = 0000$: **완전 내부**—전체 저장 (trivial accept)
- $RC_1 \text{ AND } RC_2 \neq 0000$: **완전 외부**—전체 제거 (trivial reject)
- 그 외: 영역 코드가 0 이 아닌 끝점 기준으로 경계와의 교점을 계산해 자르고, 영역 코드를 다시 계산하여 반복

비트 연산으로 자명한 accept/reject 를 빠르게 걸러내는 것이 핵심 최적화이다.

(2) 리앙-바스키 (Liang-Barsky)

선분을 매개변수식 $x = x_1 + \alpha(x_2 - x_1)$, $y = y_1 + \alpha(y_2 - y_1)$, $0 \leq \alpha \leq 1$ 으로 표현하고, 4 개 경계에 대해 $\alpha p_k \leq q_k$ 형태의 부등식을 세운다. **내부로 들어가는** 경계 ($p_k < 0$) 의 α 중 최댓값을

$\alpha_1 = \max(\alpha_{enter}, 0)$, 외부로 나가는 경계 ($p_k > 0$) 의 α 중 최솟값을 $\alpha_2 = \min(\alpha_{leave}, 1)$ 로 잡는다. $\alpha_1 > \alpha_2$ 이면 전체 제거, 아니면 $[\alpha_1, \alpha_2]$ 구간이 유효 선분이다. 교점을 반복 계산할 필요가 없어 Cohen-Sutherland 보다 대체로 효율적이다.

TA 노트 두 알고리즘의 트레이드오프를 비교하는 문제가 시험에 잘 나옵니다: Cohen-Sutherland 는 비트 연산으로 자명한 케이스를 빠르게 걸러내지만 애매한 경우 여러 번 반복 클리핑이 필요합니다. Liang-Barsky 는 매개변수 구간 계산으로 한 번에 최종 구간을 얻지만 4 개 부등식을 항상 계산해야 합니다. "긴 선분이 많고 대부분 완전히 밖에 있는 장면" 과 "선분이 윈도 경계를 여러 번 넘나드는 장면" 중 어느 쪽이 어느 알고리즘에 유리한지 물어보면 이해도를 확인하기 좋습니다.

5.5 다각형 클리핑

서덜랜드-호즈먼 (Sutherland-Hodgman)

좌 → 우 → 하 → 상 4 개 경계에 대해 **파이프라인 방식**으로 순차 클리핑한다. 각 경계에서 변의 시작점/끝점 위치 (내부·외부) 에 따라 4 가지 케이스로 출력 꼭짓점을 결정한다:

1. 외부 → 내부: 교점 + 끝점 출력
2. 내부 → 내부: 끝점만 출력
3. 내부 → 외부: 교점만 출력
4. 외부 → 외부: 출력 없음

각 단계가 독립적으로 파이프라인화되어 병렬처리가 가능하다는 장점이 있으나, **볼록다각형에만** 정확히 동작하며 **오목다각형에서는 단순다각형이 아닌 결과** (불필요한 변이 생김) 가 나올 수 있다.

웨일러-애서튼 (Weiler-Atherton)

볼록·오목 모두 처리 가능하며, 3 차원 가시면 판정에도 활용된다 (9 장 참고). 다각형 둘레를 추적하다가 클리핑 경계를 내부 → 외부로 통과하면 경계를 따라 이동해 다음 교점을 찾는 방식으로, 오목다각형 클리핑 시 여러 개의 분리된 다각형을 올바르게 생성할 수 있다.

	알고리즘	대상	특징
6 장 한눈에 비교	Cohen-Sutherland	선분	영역 코드 비트 연산, 반복 클리핑 가능
	Liang-Barsky	선분	매개변수 구간 계산, 대체로 더 빠름
	Sutherland-Hodgman	볼록 다각형	파이프라인·병렬처리 가능, 오목 다각형 한계
	Weiler-Atherton	볼록/오목 다각형	오목다각형도 정확히 처리, 가시면 판정에도 활용

자주 하는 실수 Sutherland-Hodgman 을 오목다각형에 그대로 적용하면 "찢어진" 결과가 나오는 것을 실습에서 직접 보여주면 학생들이 왜 Weiler-Atherton 이 필요한지 체감합니다. 과제 채점 시 오목다각형 테스트 케이스를 반드시 하나 포함시키는 것을 권장합니다.

6 3차원 뷰잉 (7 장)

6.1 뷰잉 파이프라인과 카메라 비유

3D 뷰잉은 사진 촬영 과정에 비유된다: 삼각대 위치 (뷰 지점) → 카메라 방향 (관찰 방향) → 사진의 위쪽 (뷰업 벡터) → 줌 (투영 파라미터) → 셔터 (래스터화). 파이프라인:

모델링좌표 → 세계좌표 → 뷰잉좌표 → 투영변환 → 정규화좌표 (+ 클리핑) → 뷰포트 → 장치좌표

2D 와 달리 클리핑 윈도가 **뷰 평면** 위에 있고, 3D 클리핑 영역인 **뷰 체적 (view volume)** 은 클리핑 윈도·투영 유형·관찰 방향에 따라 결정된다.

6.2 뷰잉 좌표 파라미터

- 뷰 지점 $P_0 = (x_0, y_0, z_0)$: 관찰자 위치, 뷰 원점.
- 뷰 평면 법선 벡터 N : 보통 관찰 방향과 나란 (z_{view} 축).
- 뷰업 벡터 V : 화면의 "위쪽" 방향 (흔히 세계좌표 y 축과 평행하게 설정).

$n = N/\|N\|$, $u = (V \times n)/\|V \times n\|$, $v = n \times u$ 로 서로 수직인 뷰잉 좌표축의 단위벡터를 구한다 (그람-슈미트 직교화와 유사). 뷰잉 변환행렬은

$$M_{VT} = R_{VT} T_{VT}, \quad T_{VT} = T(-x_0, -y_0, -z_0), \quad R_{VT} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

즉 뷰 원점을 세계 원점으로 이동 → 뷰잉 축을 세계 축과 일치시키는 회전의 두 단계 (5 장 좌표계 변환의 3D 응용) 로 유도된다.

6.3 평행투영

투영선이 서로 평행. 물체의 비례가 유지되어 CAD/공학 도면에 널리 사용.

- **직교투영**: 뷰 평면에 수직인 투영선. 정면도/측면도/평면도, 여러 축과 동시에 교차시키는 **축각 직교투영**, 세 축이 120° 로 투영되는 **등각투영 (isometric)** 포함.
- **경사투영**: 투영선이 뷰 평면과 수직이 아님. 투영각 α 가 $\tan \alpha = 1$ 이면 **캐버리어 (cavalier) 투영** (수직 변의 길이 유지), $\tan \alpha = 2$ ($\alpha \approx 63.4^\circ$) 이면 **캐비닛 (cabinet) 투영** (수직 변이 절반 길이로 표시).

직교투영 뷰 체적은 **직육면체**, 경사투영 뷰 체적은 **평행육면체**이다.

6.4 원근투영

투영기준점 (PRP) 으로 모이는 투영선을 사용, 먼 물체가 작게 보인다. 평행한 직선들이 뷰 평면과 평행하지 않으면 한 점 (**소실점**) 으로 모인다. 뷰 평면이 교차하는 주축 수에 따라 **1 점/2 점/3 점 투영** 으로 구분된다.

원근투영 변환은 동차좌표의 w (동차 파라미터) 가 z 에 의존하게 만들어, 최종적으로 각 성분을 w 로 나누는 **원근 나눗셈 (perspective division)** 이 필요하다. 이 나눗셈 때문에 **원근투영은 아핀 변환이 아니다** (5 장 경고 참고). 뷰 체적은 투영기준점을 꼭짓점으로 하는 **사각뿔**이며, 가까운/먼 클리핑 평면을 추가하면 **절두체 (frustum)** 가 된다.

대칭형 절두체는 **가시각 (field of view)** 과 **중횡비**로 정의할 수 있다:

$$\text{height} = 2(z_{prp} - z_{vp}) \tan(\theta/2), \quad \text{aspect} = \frac{\text{width}}{\text{height}}$$

가시각이 작아지면 줌인 효과 (원근감 약화), 커지면 광각 효과 (원근감 강조) 가 된다. 비대칭형 절두체는 z 축 **밀림변환 (shear)** 으로 먼저 대칭형으로 변환한 뒤 처리한다.

6.5 정규화와 뷰 체적

직교·원근 투영 모두 최종적으로 뷰 체적을 $[-1, 1]^3$ (또는 $[0, 1]^3$) 정육면체로 정규화한다. 왼손 좌표계를 스크린 좌표로 쓰면 z 증가 방향이 곧 화면 안쪽 (깊이 증가) 방향이 되어 직관적이다. 3D 클리핑은 2D 코헨-서덜랜드의 확장으로, 가까운/먼 평면 조건이 추가된 **6 비트 영역 코드**를 사용한다.

투영 방식 비교	평행투영 (직교/경사)		원근투영
	비례 뷰 체적 용도	유지됨 (아핀 변환) 직육면체/평행육면체 CAD, 공학 도면	유지되지 않음 (원근 나뉨셈) 절두체 (사각뿔대) 사실적 렌더링, 게임/영화

TA 노트 7 장은 3D 그래픽스 파이프라인 중 학생들이 가장 헷갈려하는 부분입니다. 실습에서 카메라가 이상하게 보이는 문제의 8 할은 (1) 뷰업 벡터와 관찰 방향이 평행 (외적이 0 벡터) 한 경우, (2) near/far 평면 설정이 뒤바뀌거나 0 에 너무 가까운 near 값으로 인한 z-fighting, (3) 종횡비를 뷰포트 픽셀 비율과 다르게 설정해 물체가 찌그러지는 문제입니다. OpenGL 의 `gluLookAt`/`glFrustum` 같은 고전 함수가 바로 이 장의 수식을 그대로 구현한 것이라는 점을 보여주면” 왜 이 수학을 배우는가” 에 대한 학생들의 의문이 해소됩니다. 현대 엔진 (Unity 의 Camera, Unreal 의 CameraComponent) 도 내부적으로 동일한 뷰/투영 행렬 개념을 사용합니다.

자주 하는 실수 비대칭 절두체 (예: VR 헤드셋의 좌우 눈 카메라, 타일 기반 분할 렌더링) 를 다루는 문제에서 학생들이 대칭형 공식만 암기해 적용하는 오류가 흔합니다. 시험에 낼 때는” 이 절두체가 대칭인지 먼저 확인하라” 는 문구를 명시하는 것이 좋습니다.

7 선과 면의 표현 (8 장)

7.1 3 차원 객체 모델링 방식

- **경계표현 (B-rep)**: 물체의 경계면 (다각형 메시, 스플라인 곡면) 으로 표현. 가장 기본적인 방식.
- **공간분할표현**: 정육면체를 재귀적으로 8 분할하는 **옥트리 (octree)** 등, 겹치지 않는 영역들의 집합으로 표현.
- **구조적 입체기하 (CSG)**: 기본 입체를 불 연산 (합집합/교집합/차집합) 과 기하변환으로 결합. 트리의 단말 노드는 기본 입체, 비단말 노드는 연산자.
- **스위핑 (sweeping)**: 2D 단면을 3D 공간에서 이동·회전시켜 입체 생성 (예: 회전 스위핑으로 와인 글라스 모델링).

7.2 2 차 곡면

구 $x^2 + y^2 + z^2 = r^2$, 타원면 $(x/r_x)^2 + (y/r_y)^2 + (z/r_z)^2 = 1$, 원환면 (토러스) $(\sqrt{x^2 + y^2} - r_c)^2 + z^2 = r^2$. GLUT 은 `glutWireSphere/SolidSphere`, `glutWireTorus/SolidTorus`, `glutWireCone/SolidCone` 등으로 손쉽게 생성할 수 있다.

7.3 스플라인 곡선 개요

스플라인은 몇 개의 **제어점**으로 매개변수 다항식 곡선을 정의한다. 다항식이 제어점을 모두 지나면 **보간곡선**, 근처를 매끄럽게 지나가면 (반드시 통과하지는 않음) **근사곡선**이다. 제어점을 둘러싸는 최소 다각형이 **볼록형 (convex hull)** 이며 곡선의 개략적 범위를 파악하거나 클리핑에 활용된다. 매개변수 $u \in [0, 1]$ 에 대한 3 차 다항식

$$x(u) = au^3 + bu^2 + cu + d$$

형태가 그래픽스에서 표준적으로 쓰인다.

연속성

차수	기하학적 연속성 (G^n)	매개변수 연속성 (C^n)
0	끝점 일치	끝점 일치 (동일)
1	접선 방향만 비례 (일치)	1 차 미분 벡터가 동일 (크기까지)
2	곡률 중심 동일	2 차 미분 벡터가 동일

C^n 을 만족하면 G^n 도 만족하지만 역은 성립하지 않는다. 애니메이션 경로가 G^1 만 만족하면 속도가 급변할 수 있고, C^1 을 만족하면 부드러운 속도 변화, C^2 까지 만족해야 가속도까지 매끄럽다.

7.4 에르미트 (Hermite) 스플라인

2 개의 제어점 $P(0), P(1)$ 과 그 지점의 1 차 미분 $P'(0), P'(1)$, 총 4 개의 제한요소로 정의되는 3 차 곡선.

$$P(u) = P(0)H_0(u) + P(1)H_1(u) + P'(0)H_2(u) + P'(1)H_3(u)$$

여기서 $H_i(u)$ 는 **에르미트 배합함수** (기저행렬 M_H 로부터 유도) 이다.

7.5 베지어 (Bézier) 스플라인

$n + 1$ 개의 제어점으로 정의되는 n 차 곡선. 제어점을 **선형 조합** (de Casteljau 방식) 하여 구성하며 일반식은

$$P(u) = \sum_{k=0}^n p_k \cdot BEZ_{k,n}(u), \quad BEZ_{k,n}(u) = \binom{n}{k} u^k (1-u)^{n-k} \text{ (변스타인 다항식)}$$

성질: 처음/마지막 제어점을 통과 ($P(0) = p_0, P(1) = p_n$), 양 끝 미분 $P'(0) = n(p_1 - p_0), P'(1) = n(p_n - p_{n-1})$, **볼록할 내부에 존재, 아핀 불변성** 성립 (단 원근투영 불변성은 없음). 4 개 제어점의 3 차 베지어 곡선은 끝점 미분이 $P'(0) = 3(p_1 - p_0), P'(1) = 3(p_3 - p_2)$ 인 에르미트 곡선과 동일하다.

한계: 제어점 하나를 옮기면 곡선 **전체**가 영향을 받는다 (국부 제어 불가). 이 때문에 고차 베지어보다 4 점짜리 3 차 조각을 여러 개 이어 붙이는 방식이 실무에서 선호된다.

7.6 B-스플라인

차수를 제어점 수와 독립적으로 설계자가 정할 수 있고, **국부 제어**가 가능하다. **콕스-드보어 (Cox-de Boor) 재귀식**으로 배합함수를 정의한다:

$$B_{k,0}(u) = \begin{cases} 1, & u_k \leq u < u_{k+1} \\ 0, & \text{그 외} \end{cases}, \quad B_{k,d}(u) = \frac{u - u_k}{u_{k+d} - u_k} B_{k,d-1}(u) + \frac{u_{k+d+1} - u}{u_{k+d+1} - u_{k+1}} B_{k+1,d-1}(u)$$

매개변수 구간 경계값을 **매듭 (knot)** 이라 한다.

- **균일 B-스플라인:** 매듭 간격 일정. 제어점을 근사 (통과하지 않음).
- **비균일 B-스플라인:** 매듭 간격 변화 → 곡선 형태 제어. **매듭 중복**은 곡선을 해당 제어점 쪽으로 끌어당기며, 중복 수만큼 연속성이 낮아진다 (d 차 곡선이 d 번 중복되면 그 지점에서 곡선이 제어점을 통과).
- **개방 (open) B-스플라인:** 앞뒤 매듭을 $d + 1$ 번씩 중복—양 끝 제어점을 지나게 됨. 4 개 제어점의 3 차 개방 B-스플라인은 **베지어 곡선과 동일**.

7.7 NURBS (비균일 유리 B-스플라인)

$$P(u) = \frac{\sum_k w_k p_k B_{k,d}(u)}{\sum_k w_k B_{k,d}(u)}$$

가중치 w_k 가 클수록 해당 제어점 쪽으로 곡선이 강하게 끌린다. 모든 가중치를 1로 두면 일반 B-스플라인과 같다. **원뿔곡선 (타원·포물선·쌍곡선) 을 정확히 표현할 수 있고** (3 개 제어점, 가중치 $w_1 = r/(1-r)$ 로 종류 결정), **아핀 불변성뿐 아니라 원근투영 불변성까지 성립**하는 유일한 스플라인이다—CAD 산업 표준 (Rhino, CATIA 등) 으로 채택된 이유이다.

스플라인 특성 비교		에르미트	베지어	비유리 B-스플라인	NURBS
	끝점 연결	O	O	X(근사)	X(근사)
	국부 제어	-	X	O	O
	볼록혈 성질	X	O	O	O
	원뿔곡선 재현	X	X	X	O
	아핀 불변성	O	O	O	O
	원근투영 불변성	X	X	X	O

7.8 스플라인 곡면과 테셀레이션 셰이더

쌍 3 차 베지어 곡면은 $4 \times 4 = 16$ 개의 제어점으로 $P(u, v) = \sum \sum p_{ij} BEZ_{i,3}(u) BEZ_{j,3}(v)$ 로 정의된다. NURBS 곡면도 동일한 데카르트 곱 형태로 확장된다. OpenGL 4.0+ 의 **테셀레이션 셰이더** (제어 셰이더 → 고정기능 기본요소 발생기 → 계산 셰이더) 로 GPU 에서 실시간으로 스플라인 곡면을 잘게 나누어 렌더링할 수 있다.

TA 노트 "베지어 vs B-스플라인 vs NURBS" 는 실무 면접에서도 자주 묻는 개념입니다. 핵심 차별점 세 가지로 요약해서 가르치면 학생들이 오래 기억합니다: (1) **국부 제어 가능 여부** (베지어 = 불가, B-스플라인 이상 = 가능), (2) **원근투영 불변성** (NURBS 만 성립—그래서 CAD 가 NURBS 를 표준으로 쓴다), (3) **원뿔곡선 (원, 타원) 을 정확히 그릴 수 있는가** (NURBS 만 가능—일반 스플라인은 다각형으로 근사할 뿐). 폰트 (TrueType 은 이차 B-스플라인 유사 곡선, PostScript/OpenType 은 3 차 베지어) 도 이 장의 곡선이 실무에 쓰이는 좋은 예시입니다.

자주 하는 실수 연속성 표 (위) 에서 G^1 과 C^1 을 같은 것으로 혼동하는 학생이 많습니다. "접선의 방향만 같은가, 크기까지 같은가" 라는 한 문장으로 구분해서 반복 설명하는 것이 효과적입니다. 또한 베지어 곡선이 "제어점을 항상 통과한다" 고 잘못 암기하는 경우가 많은데, **처음과 끝 제어점만** 통과한다는 점을 명확히 짚어야 합니다.

8 가시면 결정 (9 장)

8.1 개념

관찰자에게 보이는 표면을 식별하는 기술이 **가시면 결정 (visible surface determination)**, 역으로 보이지 않는 면을 걸러내는 것이 **은면 제거**이다.

- **이미지공간 기법**: 투영 평면의 각 픽셀 단위로 가시성 결정 (픽셀 수에 비례한 처리시간, 메모리 소요). 예: 깊이 버퍼, A-버퍼.
- **객체공간 기법**: 3 차원 객체의 공간적 순서로 가시면 판별. 객체 수가 적거나 서로 겹치지 않을 때 효율적. 예: 깊이정렬 (화가의 알고리즘).

8.2 후면 제거 (back-face culling)

평면 $Ax + By + Cz + D = 0$ 의 법선 벡터 $N = (A, B, C)$ 방향을 바라보는 면이 **전면**, 반대가 **후면**이다. **OpenGL 기본 규칙: CCW 로 나열된 면이 전면** (`glFrontFace(GL_CCW)` 가 기본값). 후면은 전체 면의 약 절반을 차지하므로, 관찰자를 향한 후면을 미리 걸러내면 이후 가시면 결정 계산량이 크게 줄어든다. 정규화된 왼손좌표계에서는 법선의 z 성분 부호로, 혹은 투영된 다각형의 신발끈 공식 부호로 전/후면을 판별할 수 있다.

```
glEnable(GL_CULL_FACE); glCullFace(GL_BACK); (GL_BACK 이 기본값)
```

8.3 깊이 버퍼 (Z-버퍼) 알고리즘

이미지공간 기법의 대표. 프레임 버퍼와 별도로 각 픽셀의 최소 깊이를 저장하는 **깊이 버퍼**를 사용한다.

```
for all (x,y): FrameBuffer=배경색, DepthBuffer=1.0
for all 다각형 p:
    for all (x,y) in p:
        z 계산
        if z < DepthBuffer(x,y):
            색상 계산; FrameBuffer(x,y)=색; DepthBuffer(x,y)=z
```

평면 방정식에서 다음 픽셀의 z 는 증분으로 계산 가능하다: $z_{next} = z - A/C$ (한 픽셀 이동 시 상수 A/C 만큼 감소). **장점**: 다각형을 임의 순서로 처리해도 되어 구현이 단순. **단점**: 화면 해상도가 커질수록 픽셀당 계산·메모리 비용 증가. 확장판인 **A-버퍼**는 깊이 외에 색상·투명도·안티에일리어싱 정보까지 저장 (여러 표면이 겹치면 연결 리스트로 관리).

8.4 주사선 (scan-line) 알고리즘

행 단위로 가시성을 검사하는 이미지공간 기법. 4 장의 선분 테이블/활동 선분 목록 개념을 확장해, 각 주사선에서 다각형의 **On/Off 상태 전이**를 관리한다. 여러 다각형이 동시에 On 이면 z 좌표를 비교해 더 가까운 쪽을 가시면으로 채택한다. **한계**: 다각형이 서로 **교차**하는 경우 올바르게 처리되지 않으므로 교차하는 다각형은 미리 분할해야 한다.

8.5 깊이정렬 (화가의 알고리즘)

객체공간 기법. 표면을 깊이값의 **내림차순**으로 정렬해, 먼 표면부터 가까운 표면 순으로 래스터화 (마치 화가가 배경부터 덧칠하듯) 한다. 단순 정렬만으로는 부족할 수 있어, 두 표면의 z 범위가 겹치는 경우 다음 4 가지 중 하나라도 참이면 순서를 바꾸지 않아도 된다:

1. x - y 투영 경계상자가 겹치지 않는다.
2. 검사 대상 표면이 비교 표면을 포함하는 평면의 **완전히 뒤에** 있다.
3. 비교 표면이 검사 대상 평면의 **완전히 앞에** 있다.
4. 투영된 두 다각형이 실제로 겹치지 않는다 (선분 교차 검사).

넷 다 거짓이면 두 표면의 순서를 맞바꾸는 **재정렬**이 필요하다.

	알고리즘	공간	특징	비고
가시면 결정 알고리즘 비교	후면 제거	객체	법선·CCW 판별로 절반 정도 사전 제거	다른 기법과 병행 사용
	깊이 버퍼	이미지	픽셀별 최소 깊이 비교, 순서 무관	GPU 하드웨어 표준
	주사선	이미지	활동 선분 목록 활용, 행 단위 처리	교차 다각형 취약
	깊이정렬	객체	먼 것부터 그림 (화가 알고리즘)	투명 물체 렌더링에 여전히 사용

8.6 OpenGL 관련 함수

- `glFrontFace(GL_CCW|GL_CW)` —전면 판정 와인딩 순서
- `glEnable(GL_CULL_FACE); glCullFace(GL_BACK|GL_FRONT|GL_FRONT_AND_BACK);`
- `glutInitDisplayMode(GLUT_DEPTH | ...)` —깊이 버퍼 사용 선언
- `glEnable(GL_DEPTH_TEST); glClearDepth(1.0); glClear(GL_DEPTH_BUFFER_BIT);`

TA 노트 Z-버퍼는 현대 실시간 GPU 렌더링의 사실상 표준입니다 (하드웨어에 내장). 반면 화가의 알고리즘 (깊이정렬) 은 **반투명 물체의 알파 블렌딩**처럼 Z-버퍼만으로 해결 안 되는 상황 (뒤에서 앞으로 그려야 올바른 블렌딩 결과가 나옴) 에서 지금도 사용됩니다—이 연결고리를 짚어주면 ”옛날 알고리즘을 왜 배우나” 라는 질문에 좋은 답이 됩니다. 후면 제거는 폴리곤 수를 즉시 절반으로 줄이는 GPU 성능 최적화의 기본이며, 프러스텀 컬링·오클루전 컬링과 함께 실시간 렌더링 최적화의 3 대 축으로 소개하면 좋습니다.

자주 하는 실수 과제에서 `glEnable(GL_DEPTH_TEST)` 는 켜지만 `glutInitDisplayMode` 에 `GLUT_DEPTH` 를 빠뜨려 깊이 버퍼 자체가 생성되지 않아 은면 제거가 동작하지 않는 실수가 매우 흔합니다. 또한 `glClear` 호출 시 `GL_DEPTH_BUFFER_BIT` 를 빠뜨려 이전 프레임의 깊이값이 누적되는 버그도 자주 발생합니다. 물체 앞뒤 순서가 뒤섞여 보이면 이 두 가지부터 확인하도록 안내하세요.

9 조명 및 표면 렌더링 (10 장)

9.1 표면 렌더링 개요

단색으로만 채우면 입체감이 사라진다. 조명·그림자·재질을 반영해야 사실적인 렌더링이 가능하다. 다각형 메시로 곡면을 근사할 때, 인접 다각형 경계에서 기울기가 급변하며 나타나는 시각 효과가 **마하밴드 (Mach band)** 현상이다—실제로는 각 면이 균일한 밝기여도 경계 부분이 더 밝거나 어둡게 보인다. 이를 완화하기 위해 구로/퐁 음영법이 개발되었다.

9.2 조명 모델

전역조명 모델: 광원에서 물체로 직접 오는 빛뿐 아니라 **다른 물체에 반사되어 도달하는 빛까지** 고려 (사실적이지만 계산량 많음, 레이트레이싱/래디오시티가 대표적). **지역조명 모델**: 직접광만 계산 (빠르지만 그림자 진 영역이 완전한 검정으로 나타남—이를 보완하는 것이 주변광).

9.3 광원의 종류

- **주변광 (ambient light)**: 모든 면에 균일하게 가해지는 것으로 근사한 배경광. I_{amb} 로 표현.
- **점광원 (point light)**: 한 점에서 모든 방향으로 방사 (전방향성, positional). 거리 d 에 따른 광도 감쇠함수

$$f_{radatt}(d) = \frac{1}{a_0 + a_1d + a_2d^2}$$

을 사용해 급격한 감쇠 문제를 완화한다.

- **방향성 광원 (directional light)**: 태양처럼 매우 먼 광원—평행광선으로 근사, **거리 감쇠 없음**.
- **스포트라이트**: 점광원을 특정 방향 기준 발산각 θ_l 이내로 제한. 각 감쇠는

$$f_{angatt}(\alpha) = (\vec{v}_{obj} \cdot \vec{v}_{light})^{n_l} = \cos^{n_l} \alpha, \quad 0 \leq \alpha \leq \theta_l$$

9.4 표면 반사

- **난반사 (diffuse reflection, 람베르트 반사)**: 거친 표면에서 입사광이 모든 방향으로 고르게 흩어짐—보는 각도와 무관하게 밝기 동일. 입사각 θ (법선 N , 광원 방향 L 의 내적)에 비례:

$$I_{diff} = k_d I_l (N \cdot L)$$

- **정반사 (specular reflection)**: 매끄러운 표면의 하이라이트. **퐁 (Phong) 반사 모델**은 반사 세기 감쇠를 $\cos^{n_s} \alpha$ 로 모델링한다 (α : 반사벡터 R 과 시선벡터 V 사이 각, n_s : **광택계수 (shininess)**—클수록 하이라이트가 작고 강함):

$$I_{spec} = k_s I_l (R \cdot V)^{n_s}$$

9.5 퐁 (Phong) 반사 모델 종합식

주변광 + 난반사 + 정반사를 합산한 표준 지역조명식 (광원 하나 기준):

$$I = k_a I_{amb} + f_{radatt}(d) I_l [k_d (N \cdot L) + k_s (R \cdot V)^{n_s}]$$

여러 광원이 있으면 각 광원의 기여분을 합산한다. k_a, k_d, k_s 는 각각 주변광·난반사·정반사 반사계수 (재질 특성), N, L, R, V 는 모두 단위벡터이다.

벡터 요약 N : 표면 법선 L : 표면 → 광원 방향 R : 반사 방향 (L 을 N 기준으로 반사) V : 표면 → 관찰자 방향

9.6 다각형 음영법

음영법	계산 방식	특징
균 일 음 영 법 (constant / flat shading)	다각형 하나에 대해 중심에서 1회 계산, 전체를 같은 색으로 채움	가장 빠르나 마하밴드로 경계가 두드러짐
구 로 음 영 법 (Gouraud shading)	꼭짓점 법선 (공유 다각형 법선의 평균) → 꼭짓점별 조명 계산 → 내부는 색상을 쌍선형 보간	부드럽지만 하이라이트가 작으면 찌꺼기나 왜곡될 수 있음
퐁 음 영 법 (Phong shading)	꼭짓점 법선을 쌍선형 보간 해 픽셀별 법선을 구한 뒤, 픽셀마다 조명 모델 재계산	가장 정확·하이라이트 재현 우수, 계산 비용 최대

꼭짓점 법선 벡터는 그 꼭짓점을 공유하는 인접 다각형 법선의 평균으로 구한다: $N_v = \frac{\sum N_i}{\|\sum N_i\|}$. 구로 음영법은 색상을, 퐁 음영법은 **법선 벡터**를 보간한다는 차이를 명확히 구분해야 한다.

TA 노트 "구로 vs 폼" 명칭 자체가 학생들에게 혼란을 줍니다—둘 다 사람 이름 (Henri Gouraud, Bui Tuong Phong) 이며, **폼 셰이딩** (보간 대상: 법선) 과 **폼 반사 모델** (정반사 계산식) 은 서로 다른 개념인데 이름이 같아 자주 혼동됩니다. "폼 셰이딩은 각 픽셀에서 폼 반사 모델을 사용해 색을 계산하는 것" 이라고 관계를 명시적으로 짚어주세요. 실무에서는 대부분의 실시간 엔진이 폼 셰이딩 (정점 대신 픽셀 단위로 법선을 보간·조명 계산) 을 기본으로 쓰며, 이보다 더 진보한 물리기반 렌더링 (PBR, BRDF 기반) 이 현재 게임/영화 산업 표준이라는 점을 언급하면 이 장이 왜 여전히 기초로 다뤄지는지 설명할 수 있습니다 (폼 모델의 k_d, k_s, n_s 가 PBR 의 albedo/roughness/metallic 개념의 전신입니다).

자주 하는 실수 과제에서 정반사 하이라이트가 "각지게" 보이는 오류는 대개 구로 음영법에서 하이라이트가 다각형 중앙 (꼭짓점이 아닌 곳) 에 위치할 때 발생하는 **알려진 한계** (꼭짓점에서 하이라이트가 없으면 보간으로 살아나지 않음) 이지 구현 버그가 아닐 수 있습니다. 채점 시 "폼 음영법으로 바꾸면 해결되는가" 를 확인해 알고리즘 선택 문제인지 구현 버그인지 구분해야 합니다. 또한 법선 벡터를 정규화하지 않고 보간해 밝기가 균일하지 않게 나오는 실수도 흔하므로, 보간 후 반드시 재정규화하는지 확인하세요.

10 TA 실무 가이드

앞의 각 장 안에 있는 “TA 노트”와 “자주 하는 실수” 박스가 챕터별 세부 포인트라면, 이 장은 강의 운영 전체를 관통하는 실무 가이드를 다룬다.

10.1 실습 (OpenGL) 지도 시 체크리스트

1. **초기화 순서:** `glutCreateWindow` → `glewInit` → 셰이더 컴파일/링크 → VBO 생성 순서를 지키지 않으면 알아보기 힘든 오류가 난다. 학생이 “화면이 그냥 까맣다”고 하면 이 순서부터 확인한다.
2. **셰이더 오류 로그 습관화:** `glGetShaderInfoLog`/`glGetProgramInfoLog`를 출력하지 않고 붙여넣기 코드로 실습하는 학생이 많다. 첫 실습에서 “일부러 문법 오류를 내고 로그를 읽어보는” 연습을 시키면 이후 디버깅 속도가 크게 향상된다.
3. **행렬 저장 순서와 transpose 인자:** `glUniformMatrix4fv`의 `GL_TRUE/GL_FALSE`는 행렬을 행 우선으로 만들었는지 열 우선으로 만들었는지에 달려 있다. 교재 예제 코드 (`TransMat4f`)가 행 우선이라 대부분 `GL_TRUE`를 쓰는데, 학생이 다른 라이브러리 (GLM 등)를 섞어 쓰면 이 값이 뒤집혀야 한다는 점을 짚어준다.
4. **복합변환 순서:** “물체를 회전한 다음 이동하고 싶다”는 요구를 코드로 옮길 때 $T \cdot R$ 과 $R \cdot T$ 중 어느 것이 맞는지 매번 헷갈려한다. “가장 나중에 적용될 변환을 가장 왼쪽에 쓴다”는 규칙과, 실제로 손으로 2×2 예제를 풀어보게 하는 것이 효과적이다.
5. **깊이 버퍼 활성화 누락:** `GLUT_DEPTH` 디스플레이 모드 플래그와 `glEnable(GL_DEPTH_TEST)` 둘 다 필요하다. 하나만 있으면 동작하지 않는다.
6. **좌표계·와인딩 규약 통일:** 오른손 좌표계, CCW 앞면이 교재/OpenGL 기본값이다. 학생이 임의로 좌표축을 바꾸면 후면 제거·조명 계산이 모두 어긋나므로, 과제 스케레톤 코드에서 좌표계를 고정해 배포하는 것을 권장한다.

10.2 과제·시험 채점 시 챕터별 확인 포인트

장	자주 나오는 과제/문제 유형	채점 시 반드시 확인할 것
3 장	Bresenham/중간점 원 알고리즘 직접 구현	정수 연산만 사용했는지, $ m > 1$ ·수평/수직·음의 기울기 등 경계 케이스 처리
4 장	주사선 다각형 채우기, 경계선/범람 채우기	꼭짓점을 지나는 주사선 처리, 4 방향/8 방향 연결 선택이 경계 정의와 반대인지
5 장	복합 기하변환 행렬 계산	행렬 곱 순서, 고정점 변환의 이동·변환·역이동 3 단계 누락 여부
6 장	Cohen-Sutherland/ Liang-Barsky 직접 계산	영역 코드 산출 과정, α_1, α_2 계산 시 부등식 방향
7 장	뷰잉/투영행렬 유도, 절두체 파라미터 계산	대칭/비대칭 절두체 구분, near/far 값의 부호 (왼손/오른손 좌표계)
8 장	베지어/에르미트 곡선 좌표 계산, 연속성 판별	G^n 과 C^n 혼동 여부, 제어점 통과 여부에 대한 오개념
9 장	Z-버퍼/화가 알고리즘 손계산	재정렬 4 조건을 정확히 적용했는지, 다각형 교차 케이스 처리
10 장	Phong 반사식 계산, 셰이딩 결과 비교	벡터 정규화 여부, 구로/풍 셰이딩의 보간 대상 (색 vs 법선) 구분

10.3 학생들이 가장 자주 헷갈리는 개념 Top 10

1. 동차좌표에서 $h \neq 1$ 인 경우의 의미 (원근투영에서만 실질적으로 등장)
2. 행렬 곱셈 순서—열벡터 기준으로 **오른쪽부터** 적용된다는 사실
3. G^n (기하학적 연속성) vs C^n (매개변수 연속성) 의 차이
4. 구로 셰이딩 (색 보간) vs 풍 셰이딩 (법선 보간) vs 풍 반사 모델 (정반사 공식) —이름은 겹치지만 서로 다른 개념
5. 원근투영이 아핀 변환이 **아니라는** 사실과 그로 인한 NURBS 의 필요성
6. CCW/CW 와인딩 순서와 전면/후면의 관계
7. 홀-짝 규칙에서 꼭짓점을 지나는 경계선 처리
8. 왼손좌표계·오른손좌표계 전환 시점 (세계좌표는 보통 오른손, 정규화 후 스크린은 왼손 관례)
9. z -버퍼 알고리즘이 다각형 처리 순서에 무관하게 동작하는 이유 (픽셀별 최솟값 비교)
10. 베지어 곡선이” 모든” 제어점을 지난다는 오개념 (처음·끝 제어점만 통과)

10.4 현업·최신 그래픽스와의 연결 고리

이 교재는 OpenGL 고정기능 이후 세대 (셰이더 기반) 를 다루지만 기본 개념은 지금도 그대로 유효하다. 강의에서 다음과 같이 연결지어 설명하면 학생들의 동기부여에 도움이 된다.

- **그래픽스 파이프라인 (1 2 장)**: 정점 → 테셀레이션 → 지오메트리 → 래스터화 → 프래그먼트 셰이더 구조는 Vulkan/Direct3D 12/Metal 등 현대 저수준 API 에도 거의 그대로 남아 있다. Unity 의 URP/HDRP, Unreal 의 렌더 파이프라인도 이 골격 위에 구축된다.
- **래스터화 알고리즘 (3 장)**: Bresenham 류 정수 연산 아이디어는 GPU 하드웨어 래스터라이저, 임베디드 디스플레이 드라이버에 여전히 쓰인다.
- **안티에일리어싱 (4 장)**: MSAA/FXAA/TAA 는 오늘날 거의 모든 게임 그래픽 설정 메뉴에 그대로 등장하는 옵션이다.
- **기하변환 (5 장)**: 썬 그래프의 모델 행렬, 스켈레탈 애니메이션의 본 (bone) 계층 변환이 모두 이 장의 복합변환 개념의 확장이다.
- **클리핑·뷰잉 (6 7 장)**: 뷰 절두체 클리핑 (frustum culling) 은 실시간 렌더링 최적화의 기본기이며, VR 헤드셋의 비대칭 절두체 렌더링도 7 장 개념의 직접적 응용이다.
- **스플라인 (8 장)**: 폰트 렌더링 (TrueType/PostScript), 애니메이션 커브 에디터 (easing), CAD/산업디자인 (NURBS 기반 Rhino, CATIA, SolidWorks) 에서 핵심적으로 쓰인다.
- **가시면 결정 (9 장)**: Z-버퍼는 GPU 하드웨어에 내장된 표준 기법이며, 화가의 알고리즘은 반투명 오브젝트의 렌더 순서 정렬 (order-independent transparency 이전 세대 기법) 에 여전히 쓰인다. 프러스텀 클리핑·오클루전 클리핑과 함께 성능 최적화 3 대 기법을 이룬다.
- **조명 (10 장)**: 풍 반사 모델은 물리기반 렌더링 (PBR/BRDF) 의 전신이며, 전역조명 모델은 실시간 레이 트레이싱 (RTX, DXR)·경로추적 (path tracing) 으로 이어진다.

TA 노트 실습 연결 예시매 챕터가 끝날 때” 이 개념이 실제로 어디에 쓰이는가” 를 1 2 분만 짚어주는 것 만으로도 학생 만족도가 크게 올라갑니다. 예: 9 장 강의 후” 게임 그래픽 옵션에서 봤던 오클루전 클리핑이 바로 이것의 확장입니다” 라는 한 문장이면 충분합니다.

10.5 시험·퀴즈 출제 시 유의점

- **계산 문제**는 Bresenham 픽셀 좌표, 회전행렬 계산, Liang-Barsky α 범위, Z-버퍼 다음 깊이 값처럼 손으로 여러 단계를 거쳐야 하는 문제가 많다. **부분점수 기준** (공식을 올바르게 세웠는가 / 계산 과정이 맞는가 / 최종 답이 맞는가) 을 미리 명확히 정해 채점 일관성을 확보한다.
- **개념 문제**는 교재의 장별 연습문제 (객관식) 패턴을 참고하되, 단순 암기보다” 왜 그런가” 를 묻는 서술형을 섞으면 변별력이 좋다 (예: ” 왜 Sutherland-Hodgman 은 오목다각형에서 실패하는가?”).

- **OpenGL 코드 문제**는 실제 실습에서 겪은 버그 (예: transpose 인자, GLUT_DEPTH 누락) 를 지문에 실어” 이 코드의 문제점을 찾으라” 는 형식이 실습 이해도를 잘 반영한다.
- 8 장 (스플라인) 과 5 7 장 (변환·뷰잉) 을 연결하는 통합 문제 (예: ” 이 베지어 곡선을 원근투영 하면 왜 제어점을 직접 투영한 곡선과 달라지는가”) 는 여러 장의 개념을 종합적으로 이해했는지 확인하는 데 유용하다.

10.6 핵심 알고리즘 총정리 카드

알고리즘	핵심 아이디어	시간복잡도/특징
DDA	큰 축을 1 씩 증가, 작은 축은 실수 증분	구현 간단, 누적 오차 존재
Bresenham 선분	정수 판별식 부호로 다음 픽셀 결정	곱셈·실수연산 없음, 하드웨어 친화적
중간점 원	원 함수 부호 판별 + 8 분면 대칭	계산량 1/8 로 절감
홀-짜 규칙	판별선 교차 횟수의 홀/짝	꼭짓점 통과 시 특별 처리 필요
Cohen-Sutherland	4 비트 영역 코드, 비트 연산	자명한 accept/reject 빠름
Liang-Barsky	매개변수 구간 $[\alpha_1, \alpha_2]$	교차점 반복계산 불필요
Sutherland-Hodgman	4 경계 순차 (파이프라인) 클리핑	불록다각형만 정확
Weiler-Atherton	경계 추적 방식 클리핑	오목다각형·가시면 판정 겸용
Z-버퍼	픽셀별 최소 깊이 비교	순서 무관, GPU 표준
화가의 알고리즘	깊이 내림차순 정렬 후 렌더	재정렬 필요할 수 있음, 투명도에 유용
구로 셰이딩	정점 색 계산 후 쌍선형 보간	빠르지만 하이라이트 왜곡 가능
풍 셰이딩	정점 법선 보간 후 픽셀별 조명 계산	정확하지만 계산량 최대

이 노트는 교재 1 10 장 PDF 를 근거로 재구성하였으며, TA 노트/실무 연결 부분은 일반적인 실시간 그래픽스 실무 지식을 바탕으로 보완하였습니다. 강의 진도나 사용 API 버전에 따라 세부 사항은 조정하여 활용하시기 바랍니다.